

## CO2 - AT1

Name:- G. Venkata Kartiik.

Reg No:- 192471026.

Course:- Computer Architecture

Code:- CSA1205.

# Q1 2's Complement Addition and Subtraction.

Step-1  $\rightarrow$  Binary Conversion

$$+45$$

$$45 = 00101101$$

$$-23$$

$$23 = 00010111$$

$$1^s \text{ complement} \rightarrow 11101000$$

$$2^s \text{ complement} \rightarrow 11101001$$

So,

$$+45 = 00101101$$

$$-23 = 11101001$$

Step-2  $\rightarrow$  Addition  $(+45 + (-23))$

$$\begin{array}{r} 00101101 \text{ (+45)} \\ + 11101001 \text{ (-23)} \\ \hline 00010110 = 22 \end{array}$$

$$\text{Answer :- } +45 + (-23) = +22.$$

Step-3  $\rightarrow$  Subtraction  $(+45 - (-23))$

$$+45 - (-23) = 45 + 23$$

$$\begin{array}{r} 00101101 \\ 00010111 \\ \hline 01000100 = \underline{\underline{68}} \end{array}$$

$$\text{Answer :- } +45 - (-23) = +68//.$$

## Final Results

| Operation | Result |
|-----------|--------|
|-----------|--------|

$$+45 + (-23) \quad +22$$

$$+45 - (-23) \quad +68$$

overflow NO

## Q2 Carry Look - Ahead Adder (CLA)

Introduction:- A carry look ahead Adder [CLA] is a fast adder used in processors. it reduces the delay by calculating all carry bits at the same time, instead of waiting for one carry after another.

### Generate & propagate signals

For each bit:-

$$* \text{Generate } (G) = A \times B$$

$$* \text{propagate } (P) = A \oplus B$$

Where:-

→  $A$  = First input bit

→  $B$  = Second input bit

→ Generate ( $G=1$ ) :- A carry is generated.

→ propagate ( $P=1$ ) :- The input carry is passed to the next stage.

## Carry Calculation

②

The carry bits are calculated using:-

$$\rightarrow C_1 = G_0 + P_0 C_0$$

$$\rightarrow C_2 = G_1 + P_1 C_1$$

$$\rightarrow C_3 = G_2 + P_2 C_2$$

$$\rightarrow C_4 = G_3 + P_3 C_3$$

The CLA computes these carries simultaneously, making it much faster.

### Example :-

Let

$$A = 1010$$

$$B = 0110$$

Using the CLA :-

- Generate and propagate signals are calculated.
- All carries are found together.
- Final sum is obtained quickly.

### Advantages :-

- High speed addition
- Less delay
- Better processing performance
- widely used in CPUs and ALUs

## Q3 Booth's Multiplication Algorithm

Given

$$\text{Multiplicand (M)} = -6$$

$$\text{Multiplier (Q)} = +5$$

Using 4 bit binary:-

$$\rightarrow +5 = 0101$$

$$\rightarrow +6 = 0110$$

$$\rightarrow -6 = 1010 \text{ (2's complement)}$$

Initial values

$$\rightarrow M = 1010$$

$$\rightarrow -M = 0110$$

$$\rightarrow A = 0000 \text{ (Accumulator)}$$

$$\rightarrow Q = 0101 \text{ (Multiplier)}$$

$$\rightarrow Q_{-1} = 0$$

$$\rightarrow \text{Count} = 4$$

Booth's Rules

$$\rightarrow Q_0 Q_{-1} = 01 \rightarrow \text{Add } M \text{ to } A$$

$$\rightarrow Q_0 Q_{-1} = 10 \rightarrow \text{Subtract } M \text{ from } A$$

$$\rightarrow Q_0 Q_{-1} = 00 \text{ or } 11 \rightarrow \text{No operation.}$$

Steps:-

|                 |    |                                           |
|-----------------|----|-------------------------------------------|
| 1 <sup>st</sup> | 10 | $A = A - M$ , then Arithmetic Right Shift |
|-----------------|----|-------------------------------------------|

|                 |    |                                      |
|-----------------|----|--------------------------------------|
| 2 <sup>nd</sup> | 11 | No operation, Arithmetic Right Shift |
|-----------------|----|--------------------------------------|

|                 |    |                                           |
|-----------------|----|-------------------------------------------|
| 3 <sup>rd</sup> | 01 | $A = A + M$ , then Arithmetic Right Shift |
|-----------------|----|-------------------------------------------|

|                 |    |                                      |
|-----------------|----|--------------------------------------|
| 4 <sup>th</sup> | 00 | No operation, Arithmetic Right Shift |
|-----------------|----|--------------------------------------|

After 4 shifts, the final binary result is:-

$$11100010$$

This is the 8 bit 2's complement represent -30.

## Final Answer

$$-6x + 5 = -30$$

Binary Result

11100010

Decimal Result

-30

## Q4 Restoring and Non-Restoring Division

1<sup>st</sup> Given

→ Dividend = 13

→ Divisor = 3

Binary values :—

$$* 13 = 1101_2$$

$$3 = 0011_2$$

### Restoring Division

Result

$$\rightarrow \text{Quotient} = 0100_2 = 4$$

$$\rightarrow \text{Remainder} = 0001_2 = 1$$

### Non-Restoring Division

Result

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 0001_2 = 1$$

## Final Answer

$$13 \div 3 = 4$$

$$\text{Remainder} = 1$$

Both algorithms give the same result.

## Q5 IEEE 754 Floating Point Representation.

\* Convert  $-13.25$  to Binary

Decimal number

$$13.25 \rightarrow 1101.01_2 \text{ (Binary)}$$

↓

$$1.10101 \times 2^3 \text{ (Normalized)}$$

Since the number is negative:-

$$\text{sign bit} = 1$$

\* Single Precision (32-bit)

Representation

$$\rightarrow \text{sign} = 1$$

$$\rightarrow \text{Exponent} = 10000010$$

$$\rightarrow \text{Mantissa} = 101010000000000000000000$$

\* Floating point Addition.